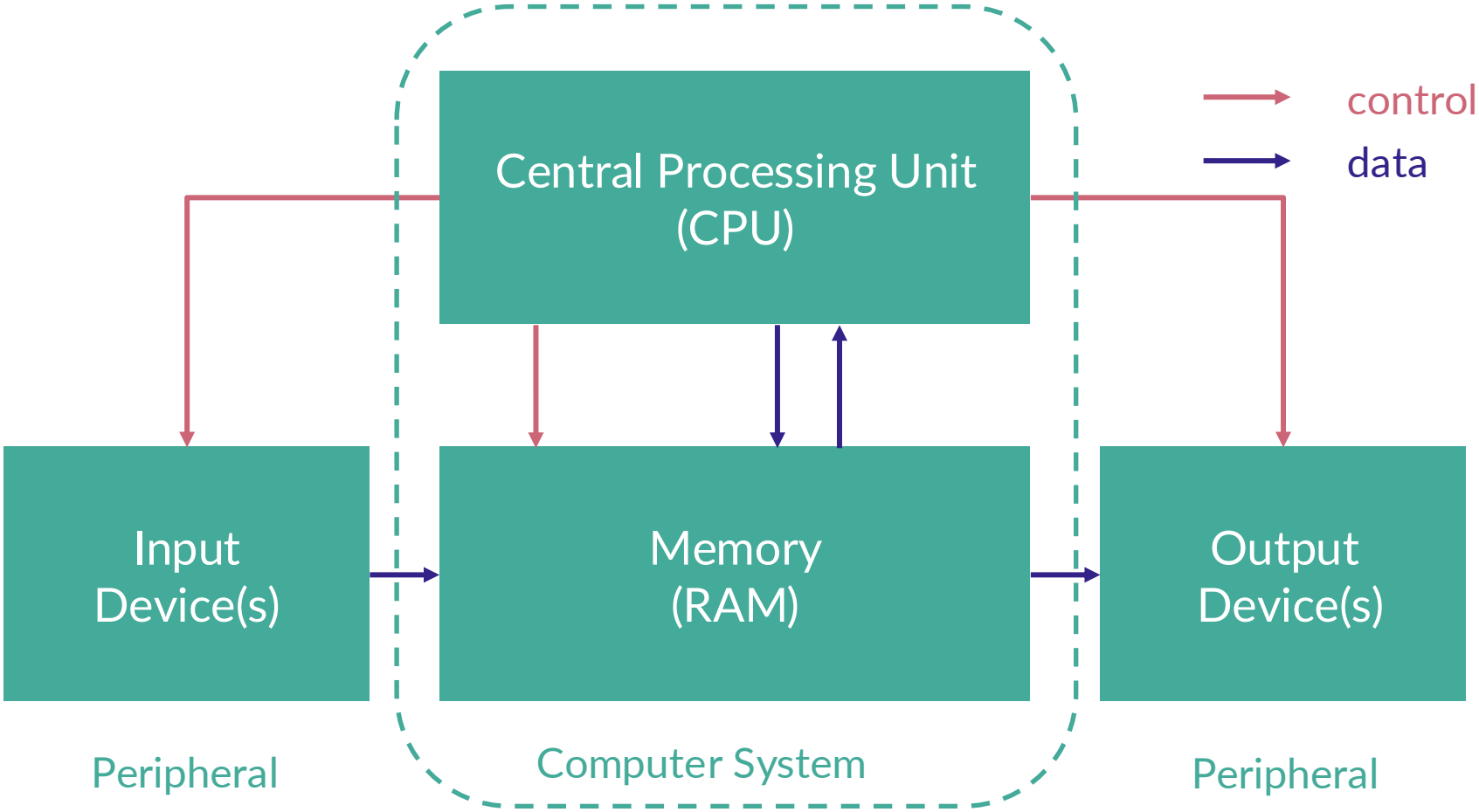
CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

Binary Representation

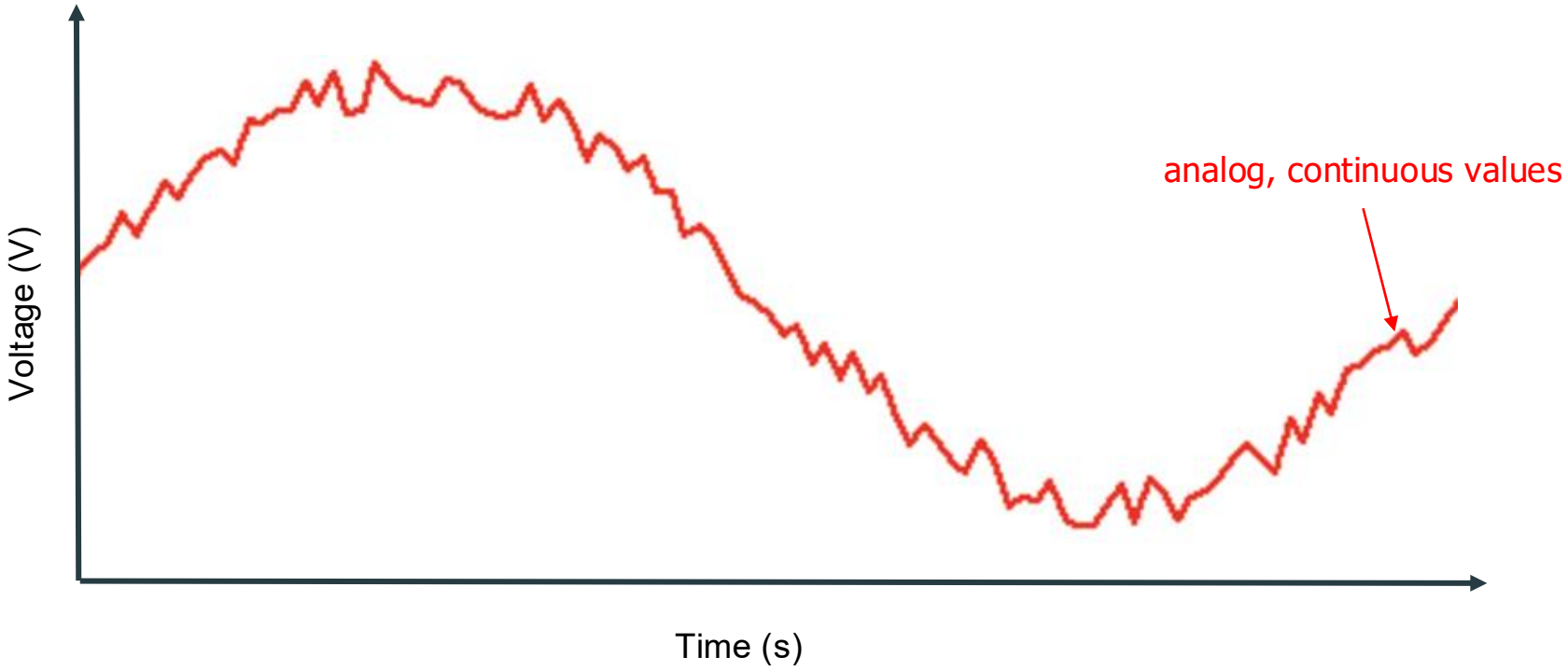
Logical Structure of a Computer

Data moves through a computer on wires.



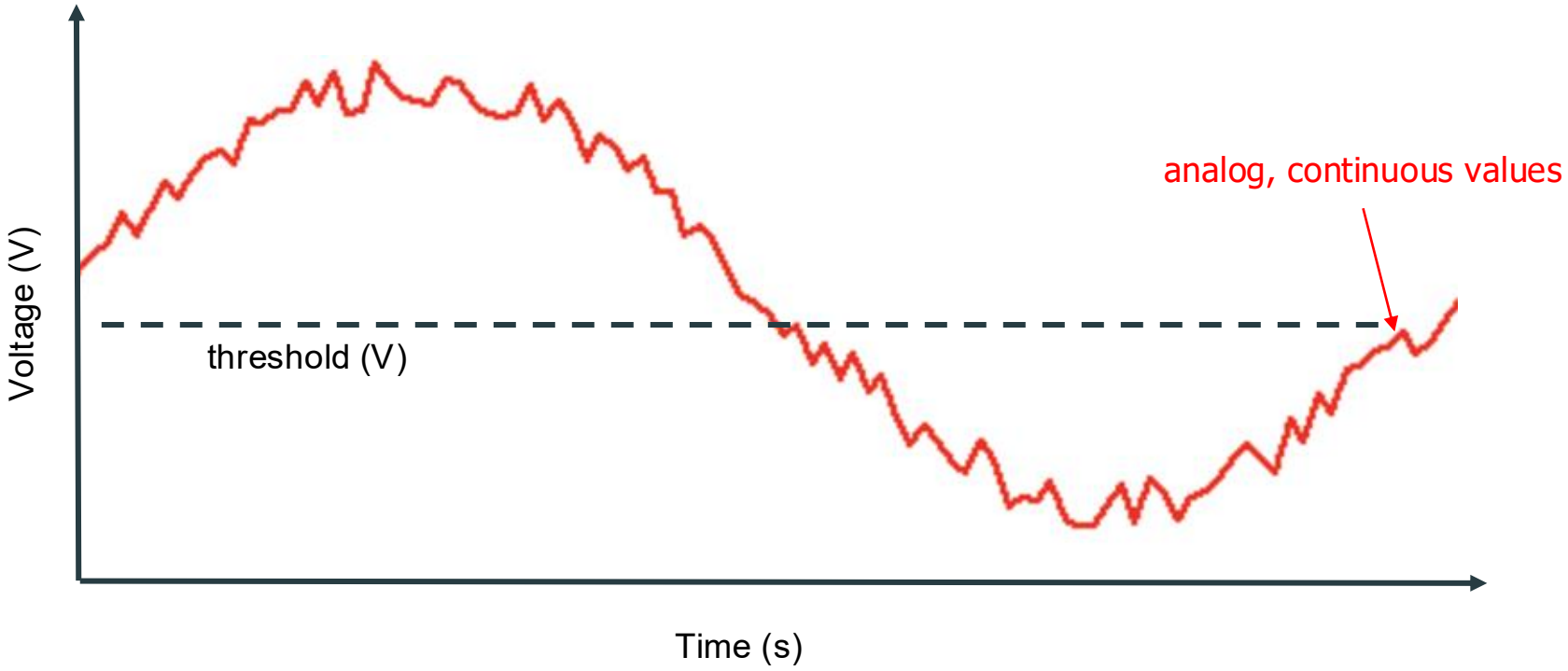
Why Do Computers Use Binary?

The binary abstraction simplifies continuous a voltage into two discrete values.



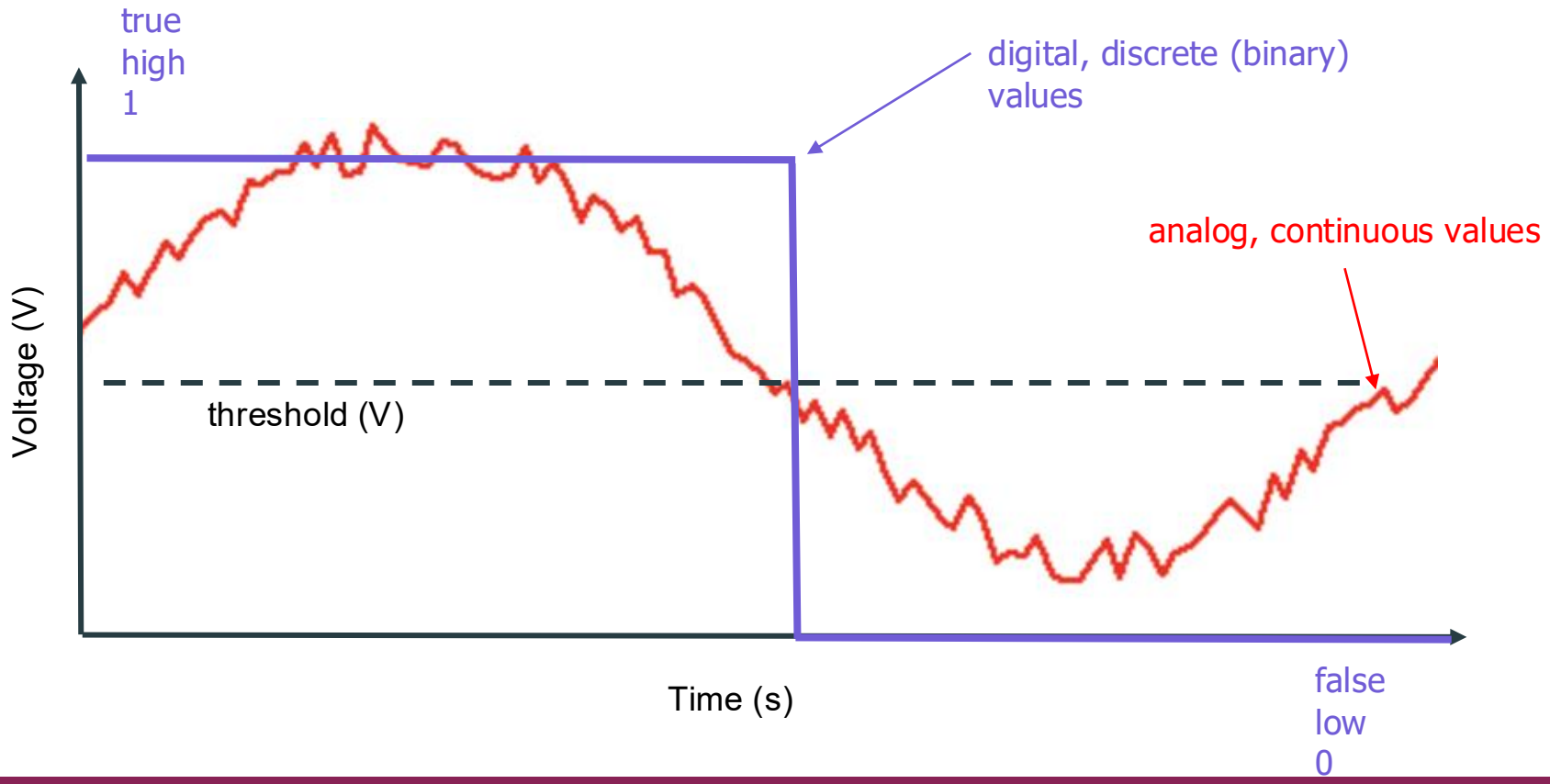
Why Do Computers Use Binary?

The binary abstraction simplifies continuous a voltage into two discrete values.



Why Do Computers Use Binary?

The binary abstraction simplifies continuous a voltage into two discrete values.



Decimal Number System

The decimal number system uses 10 *digits* to represent 10 different *values*.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

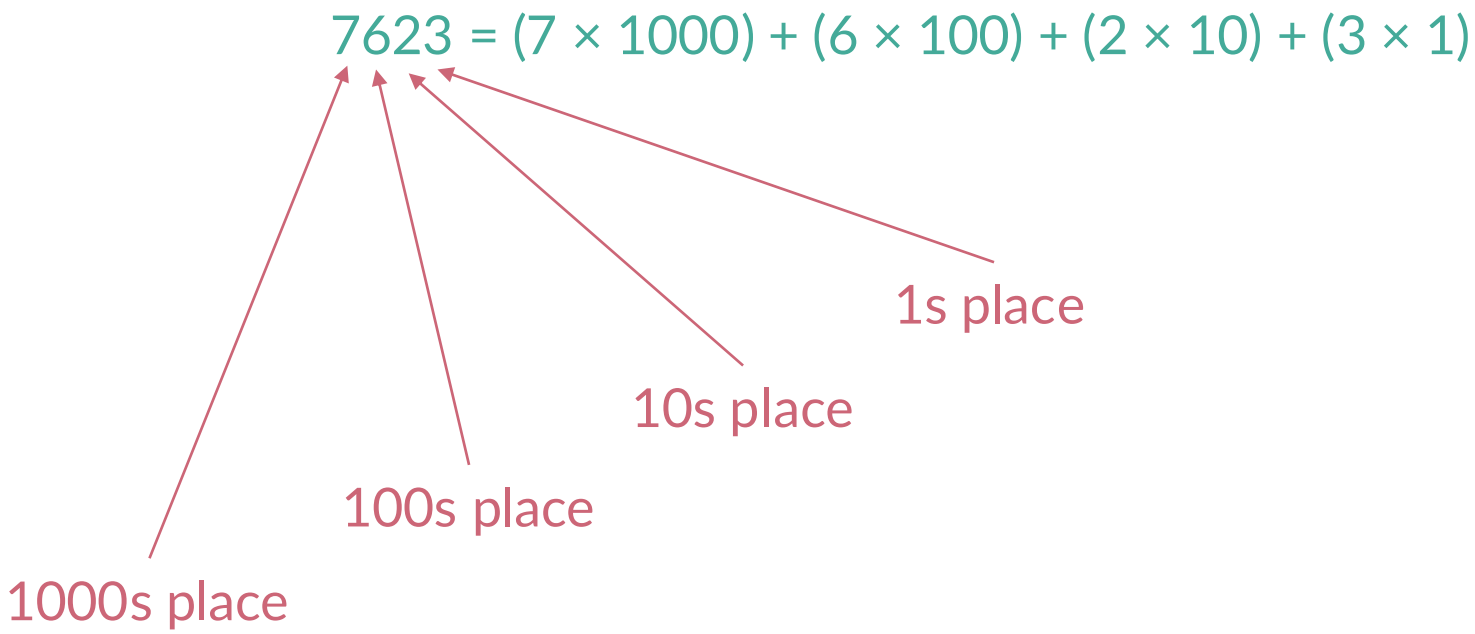
Decimal is a Positional Number System

In a positional number system, the value of a digit depends on its position within the number.

$$7623 = (7 \times 1000) + (6 \times 100) + (2 \times 10) + (3 \times 1)$$

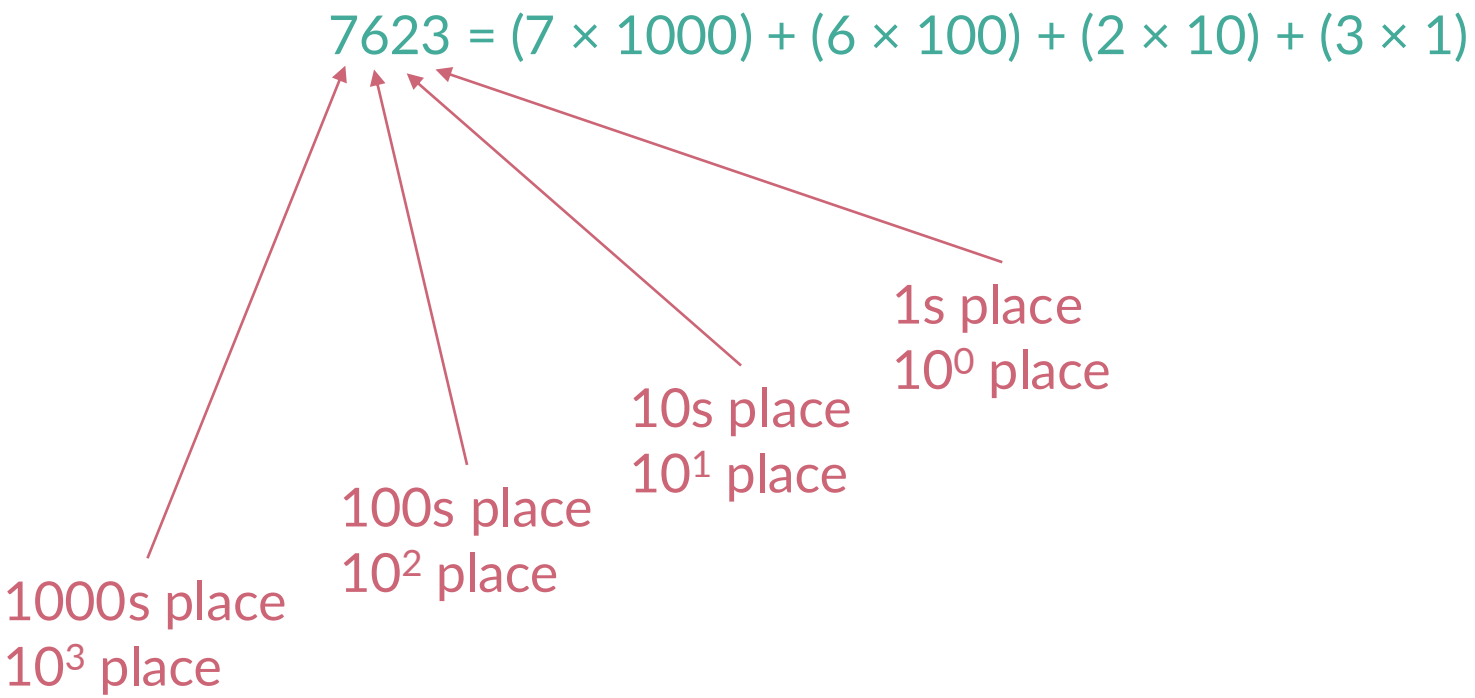
Decimal is a Positional Number System

In a positional number system, the value of a digit depends on its position within the number.



Decimal is a Positional Number System

In a positional number system, the value of a digit depends on its position within the number.



Positional Number System

In a positional number system, the value of a digit is multiplied by the base to the power of the position.

$$\ldots \; x^3 \; x^2 \; x^1 \; x^0$$

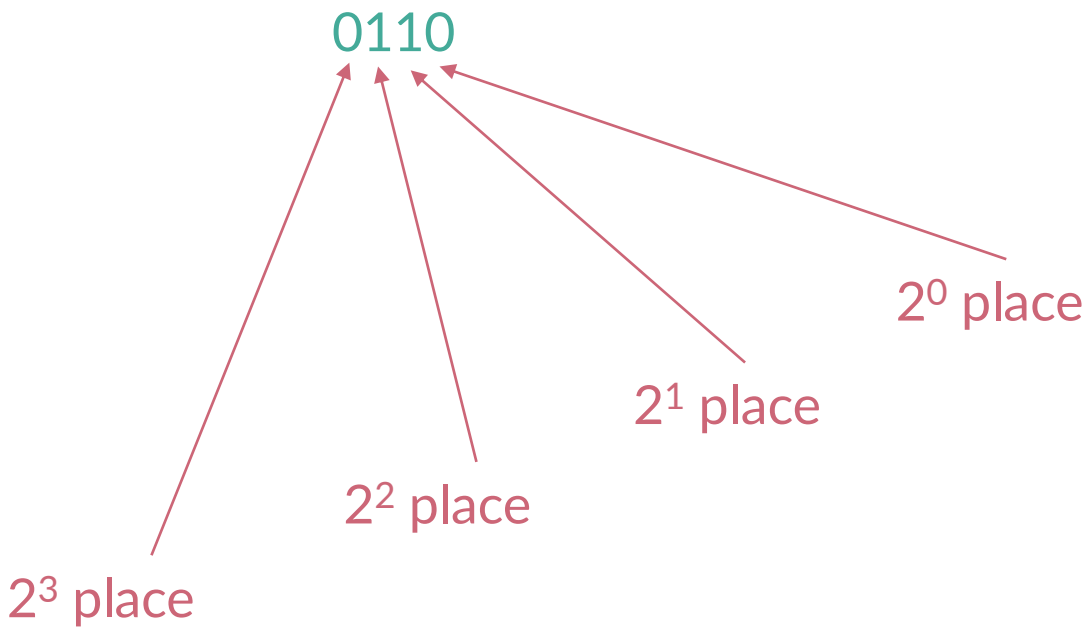
Binary Number System

The decimal number system uses 2 *digits* to represent 2 different *values*.

0, 1

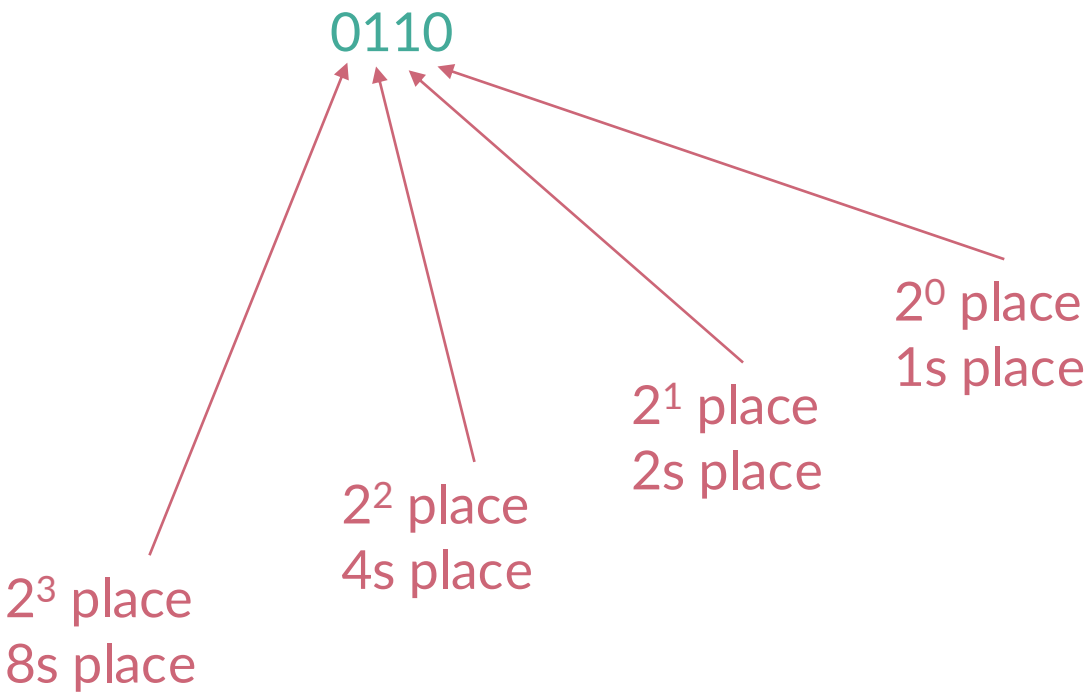
Binary is a Positional Number System

Binary is a positional number system where the base is 2.



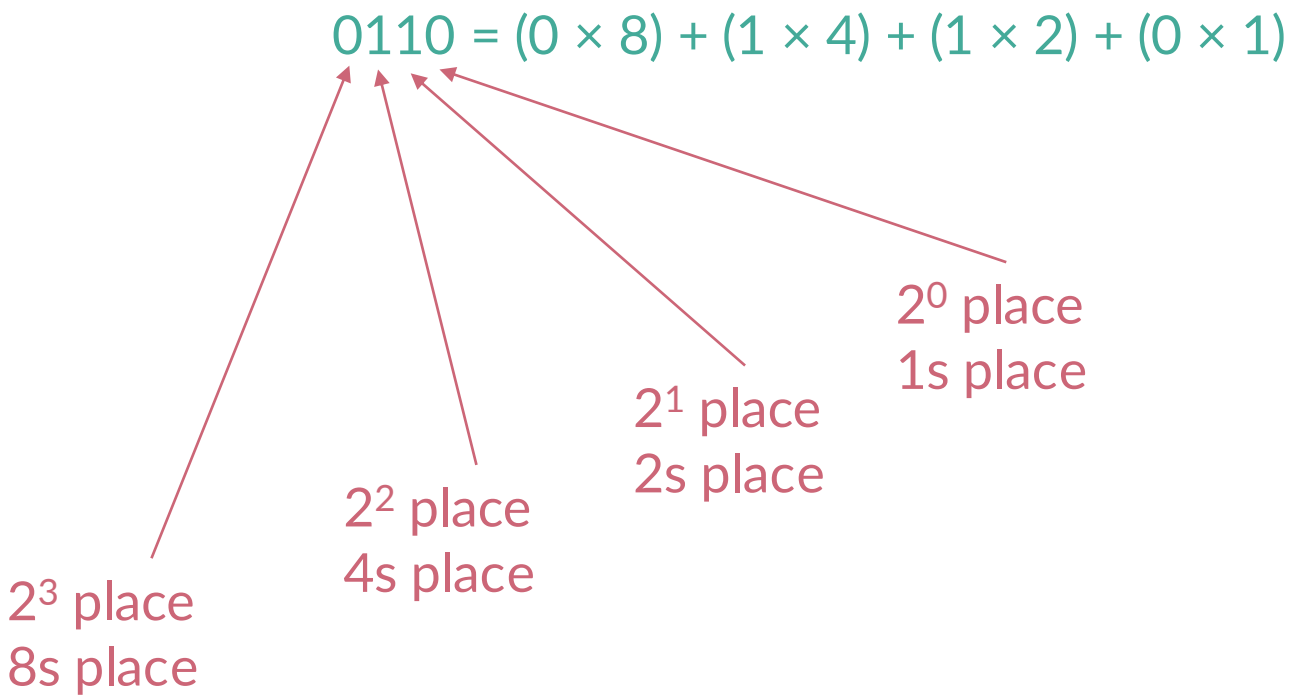
Binary is a Positional Number System

Binary is a positional number system where the base is 2.



Binary is a Positional Number System

Binary is a positional number system where the base is 2.



Vocabulary

The **most significant bit** (MSB) is the bit with the largest value, typically on the left.

The **least significant bit** (LSB) is the bit with the smallest value, typically on the right.

A **byte** is a group of 8 bits, treated as a unit.

Converting Binary to Decimal

Convert binary to decimal using place values.

$$11001010_2 = ??_{10}$$

Converting Binary to Decimal

Convert binary to decimal using place values.

$$11001010_2 = ??_{10}$$

place:	128	64	32	16	8	4	2	1
	1	1	0	0	1	0	1	0

Converting Binary to Decimal

Convert binary to decimal using place values.

$$11001010_2 = ??_{10}$$

place:	128	64	32	16	8	4	2	1
	1	1	0	0	1	0	1	0

11001010_2

$= (1\times128) + (1\times64) + (0\times32) + (0\times16) + (1\times8) + (0\times4) + (1\times2) + (0\times1)$

$= 202_{10}$

Converting Decimal to Binary

Convert decimal to binary by dividing by 2 and tracking the remainder.

$$43_{10} = ??_{2}$$

Converting Decimal to Binary

Convert decimal to binary by dividing by 2 and tracking the remainder.

$$43_{10} = ??_{2}$$

1s place	43/2	=	21	r.	1
2s place	21/2	=	10	r.	1
4s place	10/2	=	5	r.	0
8s place	5/2	=	2	r.	1
16s place	2/2	=	1	r.	0
32s place	1/2	=	0	r.	1

Converting Decimal to Binary

Convert decimal to binary by dividing by 2 and tracking the remainder.

$$43_{10} = ??_{2}$$

1s place	43/2	=	21	r.	1
2s place	21/2	=	10	r.	1
4s place	10/2	=	5	r.	0
8s place	5/2	=	2	r.	1
16s place	2/2	=	1	r.	0
32s place	1/2	=	0	r.	1

$$43_{10} = 101011_{2}$$

Adding Binary Numbers

Adding binary numbers uses the same algorithm as adding decimal numbers.

carrying: $1_2 + 1_2 = 10_2$

$$\begin{array}{rcccc} & 1 & 1 & & \\ & 0 & 1 & 1 & 0 \\ + & 1 & 0 & 1 & 0 \\ \hline 1 & 0 & 0 & 0 & 0 \end{array}$$

Adding Binary Numbers

Adding binary numbers uses the same algorithm as adding decimal numbers.

carrying: $1_2 + 1_2 = 10_2$

check your work:

			1	1	
	6		0	1	1	0
+	10	+	1	0	1	0
	16		1	0	0	0	0

Adding Binary Numbers

Adding binary numbers uses the same algorithm as adding decimal numbers.

carrying: $1_2 + 1_2 = 10_2$

check your work:

		1	1			
	6		0	1	1	0
+	10	+	1	0	1	0
	16	1	0	0	0	0

4 bits

4 bits

5 bits

the result of adding 2 n-bit numbers is up to n+1 bits

Multiplying Binary Numbers

Multiplying binary numbers uses the same algorithm as multiplying decimal numbers.

$$
\begin{array}{rcccc}
 & 0 & 1 & 1 & 0 \\
\times & & & 1 & 1 \\
\hline
 & 0 & 1 & 1 & 0 \\
+ & 0 & 1 & 1 & 0 \\
\hline
1 & 0 & 0 & 1 & 0
\end{array}
$$

Multiplying Binary Numbers

Multiplying binary numbers uses the same algorithm as multiplying decimal numbers.

check your work:

$$\begin{array}{rr} & 6 \\ \times & 3 \\ \hline & 18 \end{array}$$

$$\begin{array}{rcccc} & 0 & 1 & 1 & 0 \\ \times & & & 1 & 1 \\ \hline & 0 & 1 & 1 & 0 \\ + & 0 & 1 & 1 & 0 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

Multiplying Binary Numbers

Multiplying binary numbers uses the same algorithm as multiplying decimal numbers.

check your work:

$$\begin{array}{rr} & 6 \\ \times & 3 \\ \hline & 18 \end{array}$$

$$\begin{array}{rrrrr} & & 0 & 1 & 1 & 0 \\ & \times & & & 1 & 1 \\ \hline & & 0 & 1 & 1 & 0 \\ + & 0 & 1 & 1 & 0 & \\ \hline 1 & 0 & 0 & 1 & 0 & \end{array}$$

the result of multiplying 2 n-bit numbers is up to 2n bits

Representation is Bounded

An n bit number can represent 2^n unique values.

1 bit	2 values	0, 1
2 bits	4 values	00, 01, 10, 11
3 bits	8 values	000, 001, 010, 011, 100, 101, 110, 111
4 bits	16 values	
8 bits	32 values	
n bits	2^n values	

Overflow

Overflow occurs when the result of a computation cannot be represented with the number of bits available.

			1				
	6			0	1	1	0
+	12		+	1	1	0	0
	18			1	0	0	1	0

Overflow

Overflow occurs when the result of a computation cannot be represented with the number of bits available.

			1			
	6		0	1	1	0
+	12	+	1	1	0	0
	18	1	0	0	1	0

a 4 bit number can represent 0-15, so the result of this addition cannot be represented

Hexadecimal, or Base 16

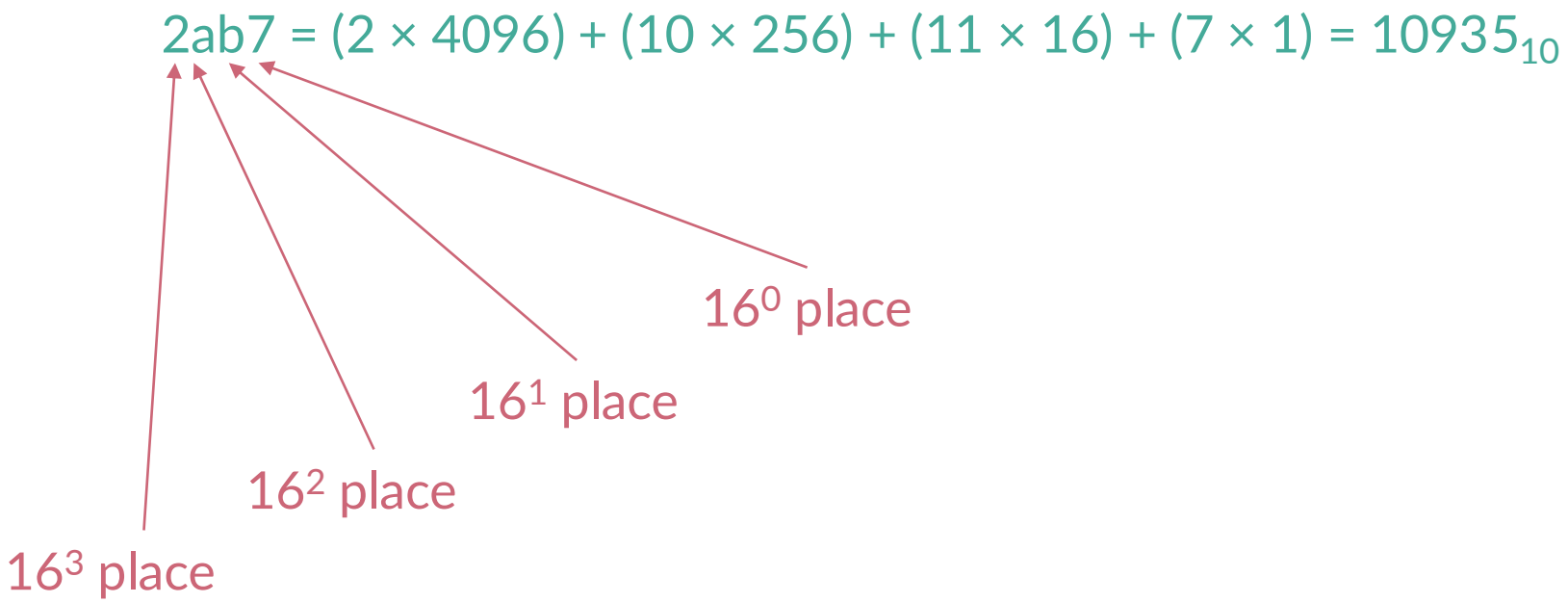
The decimal number system uses 16 *digits* to represent 16 different *values*.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

Hexadecimal, or Base 16

The decimal number system uses 16 *digits* to represent 16 different *values*.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f



Hexadecimal, or Base 16

The decimal number system uses 16 *digits* to represent 16 different *values*.

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

$2ab7 = (2 \times 4096) + (10 \times 256) + (11 \times 16) + (7 \times 1) = 10935_{10}$

16^0 place

16^1 place

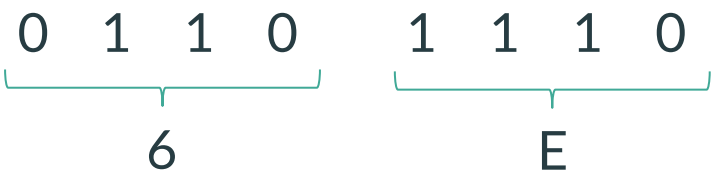
16^2 place

16^3 place

notation:
$2ab7_{16}$ = 0x2ab7

Using Hexadecimal as Shorthand

Use hexadecimal as a shorthand for binary:



01101110_2 = 0x6E

Binary	Hex	Decimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Signed Integers

Representing Signed Integers

We will use **two's complement** to represent signed integers.

- only one zero value
- easy to identify negative numbers
- add and subtract using the same algorithm

Two's Complement

In two's complement representation, the most significant bit (far left place) has a negative value and all other bits have a positive value.

two's complement $1100 = ??_{10}$

Two's Complement

In two's complement representation, the most significant bit (far left place) has a negative value and all other bits have a positive value.

two's complement 1100 = $??_{10}$

place:	8	4	2	1
	1	0	1	0

two's complement 1100

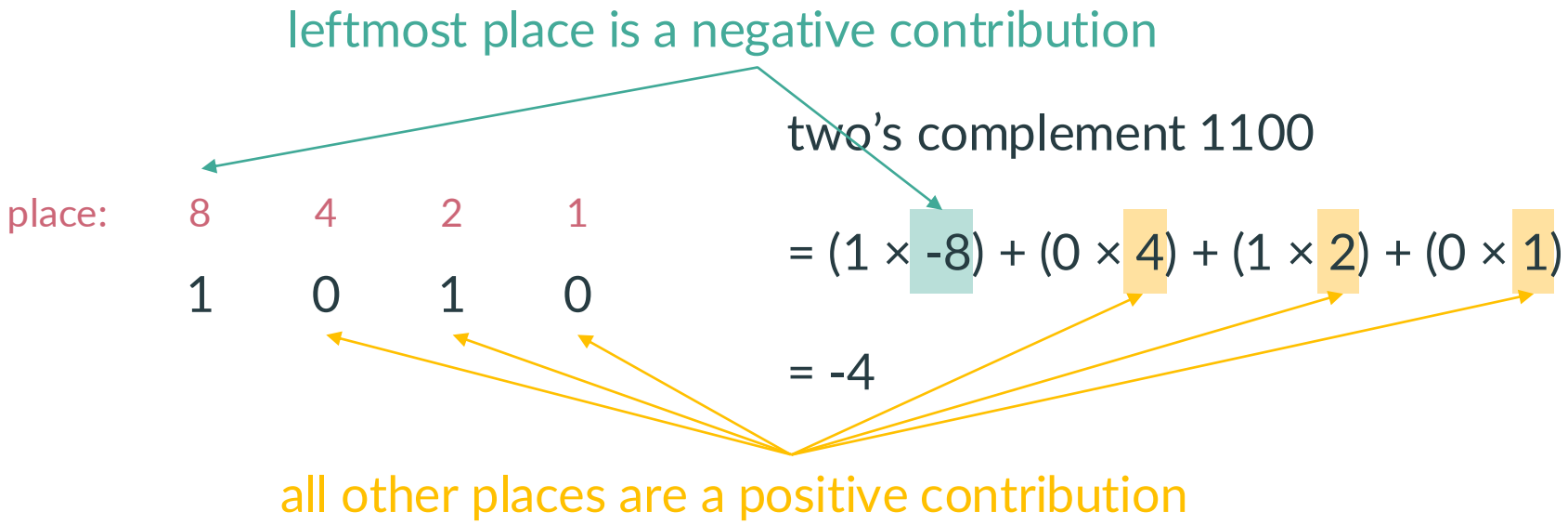
= (1 × -8) + (0 × 4) + (1 × 2) + (0 × 1)

= -4

Two's Complement

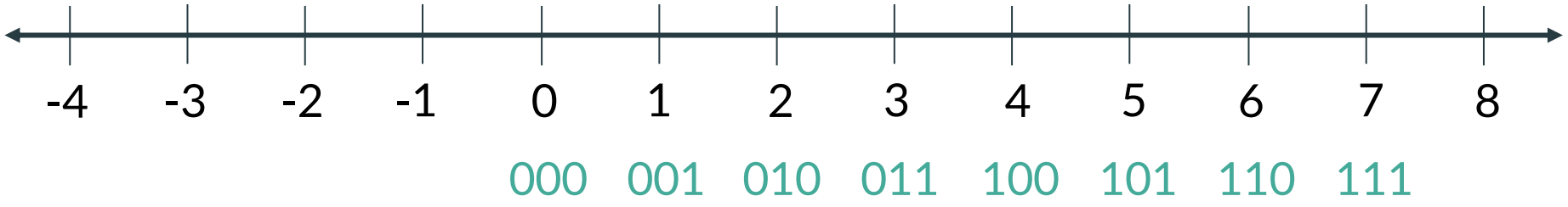
In two's complement representation, the most significant bit (far left place) has a negative value and all other bits have a positive value.

two's complement 1100 = $??_{10}$



Signed vs. Unsigned Numbers

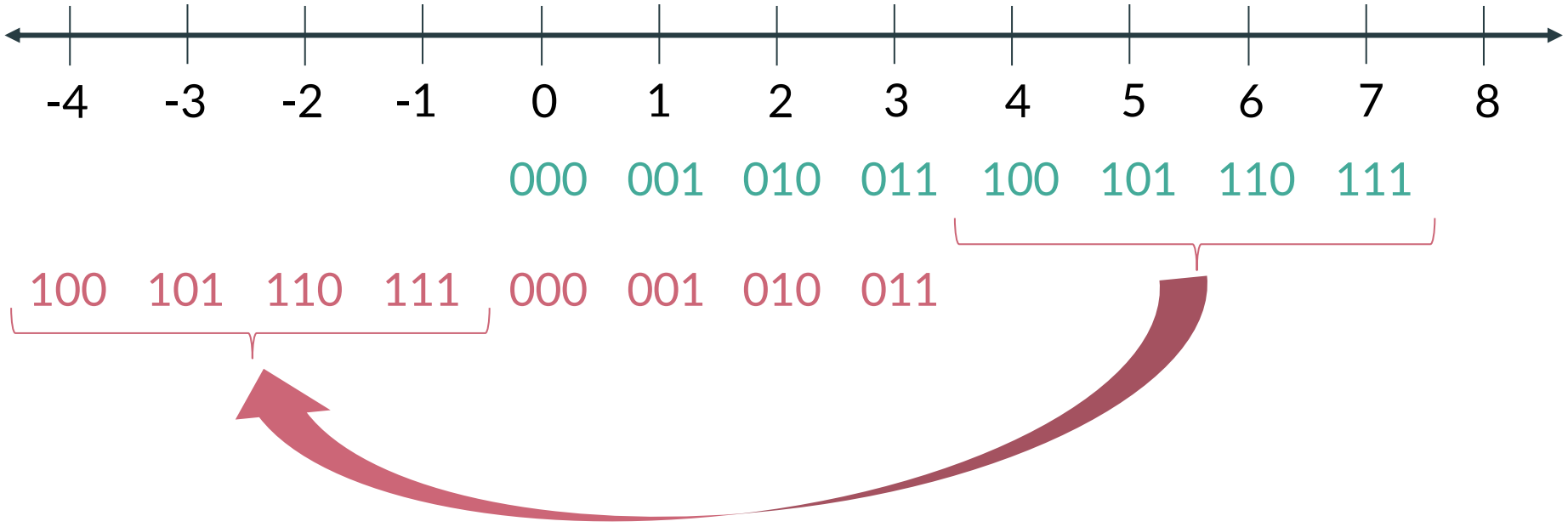
3 binary bits can represent 8 integers.



unsigned

Signed vs. Unsigned Numbers

The upper half of the range represent negative numbers in 2's complement.



Negation in Two's Complement

To negate a number in two's complement flip the bits and add one.

00101101

Negation in Two's Complement

To negate a number in two's complement flip the bits and add one.

00101101

flip the bits:
1 → 0
0 → 1

0	0	1	0	1	1	0	1
1	1	0	1	0	0	1	0

Negation in Two's Complement

To negate a number in two's complement flip the bits and add one.

00101101

flip the bits:
1 → 0
0 → 1

0	0	1	0	1	1	0	1
1	1	0	1	0	0	1	0

add one:

	1	1	0	1	0	0	1	0
+								1
	1	1	0	1	0	0	1	1

Negation in Two's Complement

To negate a number in two's complement flip the bits and add one.

00101101 = 32+8+4+1 = 45

flip the bits:
1 → 0
0 → 1

0	0	1	0	1	1	0	1
1	1	0	1	0	0	1	0

add one:

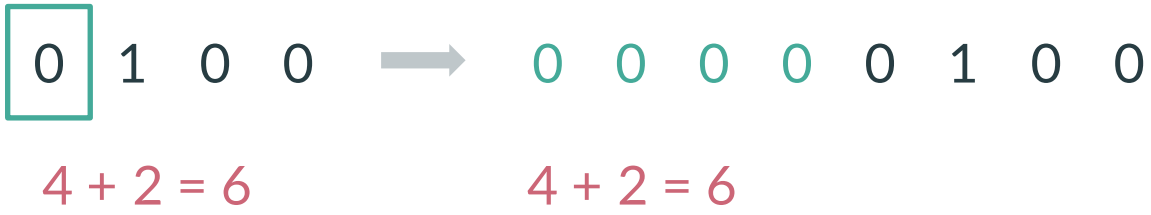
	1	1	0	1	0	0	1	0
+								1
	1	1	0	1	0	0	1	1

= -128+64+16+2+1 = -45

Sign Extension

Sign extension increases the bit width of a signed number while preserving its value.

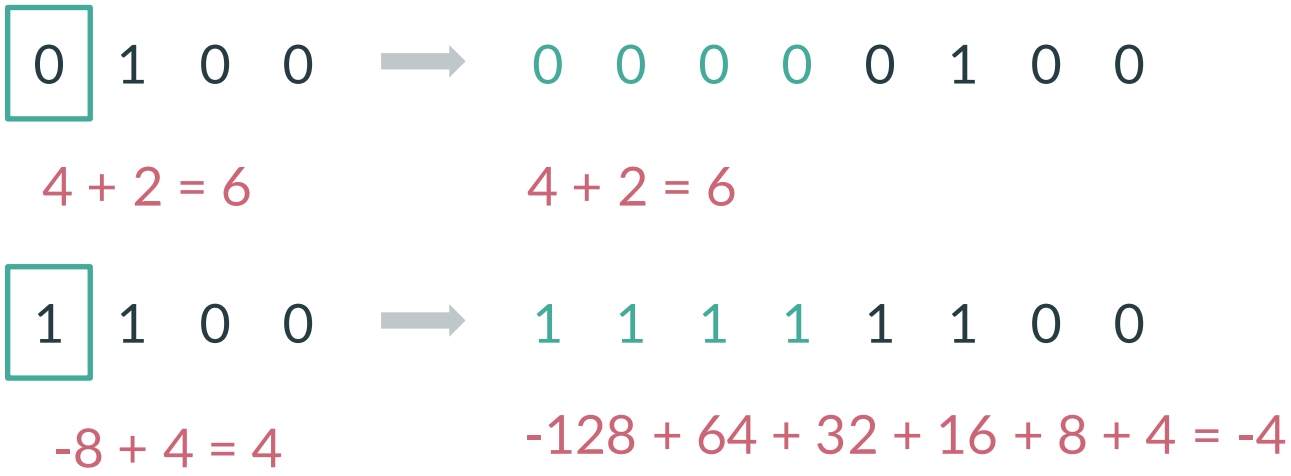
To sign extend a number, pad the empty places with the most significant bit.



Sign Extension

Sign extension increases the bit width of a signed number while preserving its value.

To sign extend a number, pad the empty places with the most significant bit.



Exercise: Binary Representation